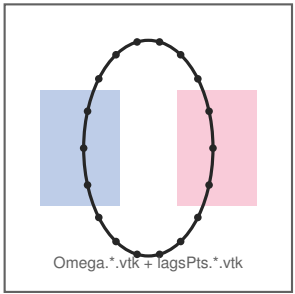
THE IB2D PRACTICAL, AND VISIT FOR IB2D

Run the rubber band in MATLAB or Python and watch it; open the `.vtk` output in VisIt step by step (points, velocity vectors, vorticity, the time slider, a movie); the forces on the structure; four more examples to run, and two to read



Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part II • Chapter 4b

Claude (Anthropic) was used to organize, combine, and format more than twelve years (2013–2026) of the instructor's tutorial notes from courses and workshops at several universities.

WHAT YOU WILL BE ABLE TO DO AFTER THIS CHAPTER

- Run the rubber-band example in MATLAB (and watch its figure) or in Python, and know what each run leaves behind.
- Open IB2d's output in VisIt from scratch: the points, the velocity vectors, the vorticity, all three together, advancing in time; make them look right (point size, limits, nodal, opacity); zoom; save a picture and a movie.
- Know the two ways IB2d's files differ from IBAMR's in VisIt (one file per dump and per variable, grouped as a "database"; MATLAB's files carry a time stamp, Python's do not), and what VisIt asks because of it (the "Index" correlation).
- Read the forces on the structure (`hier_IB2d_data`) and plot one number against time.
- Run four more Python examples (porous band, heart tube with muscles, the impedance pump with tracers, a beam with target points) and read two MATLAB ones (the swimmer, the leaf) that use the update functions of chapter 4c.

> Every VisIt step was repeated headlessly in VisIt 3.5.0 on the Python run's output (`wsl/final/visit_ch4b.sh`); most windows shown are the 2024 practical's (VisIt 3.4.1 on Windows), which look the same in 3.5; four are older VisIt 2.5.2 windows on a Mac that the practical reused (the first File open window, the zoom and the Save-movie screens). The extra examples were run in October 2026 (Python 3.12, numba 0.68); the MATLAB-only ones are described from their files and the 2024 pictures.

WHERE THIS CHAPTER COMES FROM

- “IB2d Practical” (2024, 38 slides, 25 screenshots): the run in MATLAB, the files, then VisIt 3.4.1 step by step on the rubber band (preferences, open the points, mesh, point size, open `u`, vectors, the correlation prompt, advance, vorticity, limits, play, zoom, movie), two exercises, and two bigger MATLAB examples (the swimmer, the leaf). The only VisIt-3 walk-through for IB2d; this chapter follows its order.
- “ib2d part 2” (Lecture 10, 2021, edited 2023, 32 slides): MATLAB vs Python, the “suggestions to improve” slides (limits at 25 %, nodal, 8000 vectors, constant colour, annotations off, legends, sessions, movies as PNG frames joined in VirtualDub or QuickTime), and the sawtooth that chapter 4c takes up.
- “A quick intro to ib2d” (Lecture 9), slides 31–51: the VisIt 2.5.2 walk-through written for IBAMR’s files (`lag_data.visit`, `dumps.visit`) and reused for IB2d; chapter 26 is its home.
- Nick Battista’s video tutorial 4 (VisIt for IB2d; https://youtu.be/4D4ruXbeCiQ).
- The runs of October 2026 (`wsl/final/ib2d_pyrun*.sh`, `ib2d_pump2.sh`) and the headless VisIt script `visit_ch4b.sh`.

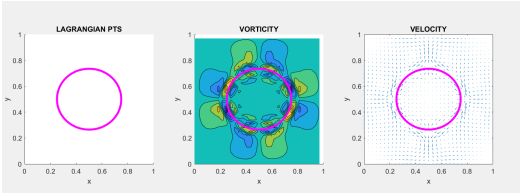
What is new here: the headless check of every step in 3.5, the time-stamp difference between the MATLAB and Python writers, the forces plot, the four Python runs with their timings, the tracer-example fix, and the pointers to chapters 26, 30, 32 and 33 for everything VisIt does beyond this walk-through.

THE RUN, IN MATLAB

Open MATLAB, go to the folder and type the name of the main file:

```
>> cd matIB2d/Examples/
     Example_Standard_Rubberband/
     Rubberband_with_Springs
>> main2d
```

- A figure window opens with three panels (`plot_Matlab = 1`): the Lagrangian points, the vorticity (filled contours) and the velocity (arrows), redrawn at every dump. The picture is the 2024 practical's: "a rubber band oscillating in a fluid".
- The console prints the banner, "FIBER MODEL INCLUDES", then `Current Time(s)` at every dump (chapter 4a).
- 1500 steps at 32×32: well under a minute, most of it the figure. Set `plot_Matlab = 0` for a batch run and look at the output in VisIt instead.



The MATLAB figure during the run (2024 practical): points, vorticity, velocity. MATLAB R2020a and R2020b cannot draw it (the README names only those two).

THE RUN, IN PYTHON; WHAT IS LEFT BEHIND

```
cd pyIB2d/Examples/Rubberband_with_Springs
python3 main2d.py            # 3.9 s on a desktop
    PC, plotting off
ls viz_IB2d | head -3        # Omega.0000.vtk ... (
    uppercase names sort first)
ls viz_IB2d | wc -l          # 456 files: 76 dumps
    x 6
ls hier_IB2d_data | wc -l    # 228: fX_Lag, fY_Lag,
     fMag per dump
```

- With `plot_Matlab = 1` the Python version opens a matplotlib window with five panels (its `input2d` has every `plot_*` flag on); on a machine without a display set it to 0 (or `MPLBACKEND=Agg`).
- The same files appear in both versions, with one difference that matters in VisIt: the MATLAB writer stamps each file with its time (VisIt shows `Time: 0.02` for dump 1), the Python writer does not (VisIt shows the state index: 0, 1, 2, . . .). Chapter 4a, slide on the output, has the file layout.

- `viz_IB2d/`: `lagsPts` (the points), `lagPtsConnect` (the points joined along the springs), `Omega`, `P`, `uMag`, `u` (vectors), and `uX`, `uY`, `fX`, `fY`, `fMag`, `tracer`, `concentration` when their flags or models are on.
- `hier_IB2d_data/`: the force on every point, as `fX_Lag`, `fY_Lag`, `fMag` (and `fNorm`, `fTan` from the MATLAB writer for structures of more than five points; the Python writer has the same branch but tests the wrong array dimension and never writes them, a third item for the note to the repository).
- Every file is a dump number: `.0000` is $t = 0$, and dump k is $t = k \times$ `print_dump` $\times \Delta t$ (0.02 s apart here).
- A second run in the same folder overwrites these files one by one; copy the folder, or delete the two output folders, before a new run.

VISIT FOR IB2D: WHAT IS DIFFERENT FROM THE IBAMR WALK-THROUGH

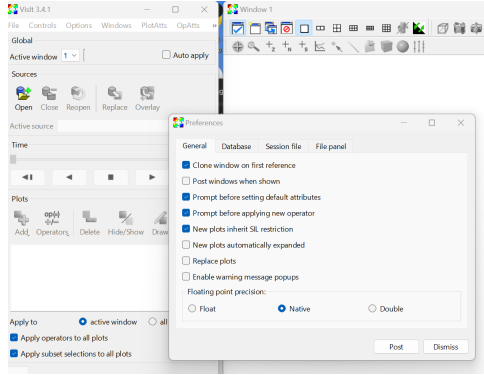
Chapter 26 opens an IBAMR run (`lag_data.visit`, `dumps.visit`: one list file each). IB2d writes one file per dump and per variable, so:

- VisIt's file panel groups the numbered files itself: **File grouping: Smart** shows `lagsPts.*.vtk database (76 total files)`, and that "database" is what you open. One database per variable: `Omega.*.vtk database`, `u.*.vtk database`, ...
- Each database has its own time slider. When a second one is added to the plots, VisIt asks **Would you like to create a "Index" database correlation for the following databases?**: say **Yes**, and the sliders move together, state by state (slide 11).
- The meshes are all called `mesh`; the scalar in the `Omega` files is `Omega`, the vector in the `u` files is `u`, the forces in `hier_IB2d_data` are `fMag`, `fX_Lag`, `fY_Lag` on the points.
- There are no AMR patches to hide, no `U_x`/`U_y` components to choose, no levels: a 32×32 uniform grid with the periodic edges.
- Everything else (plot attributes, annotations, sessions, saving windows, movies, the CLI) is the same program: chapters 26, 30 and 32 apply unchanged.

The steps below are the 2024 practical's, checked in VisIt 3.5.0.

STEP 0: INSTALL VISIT, SET ONE PREFERENCE

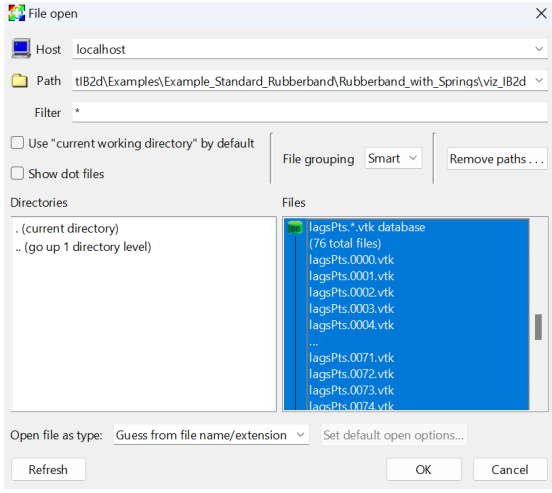
- VisIt is free (Lawrence Livermore): https://visit-dav.github.io/visit-website/; installers for Windows, macOS and Linux. Part V uses 3.5.0; the 2024 practical used 3.4.1 and the windows are the same.
- On a Mac with a recent macOS, VisIt may refuse to list files ("The MetaData server running on localhost could not get the file list for the current directory"): start it from the terminal, from the folder you want to open, with `/Applications/VisIt.app/Contents/Resources/bin/visit -gui` (the IB2d README's workaround).
- **Options** → **Preferences**: untick **Enable warning message popups**. The practical's reason: "when we start saving movies, a pop-up will spring up every time-step".



The Preferences window (3.4.1); the box to untick is the last one of the General tab.

STEP 1: OPEN THE POINTS

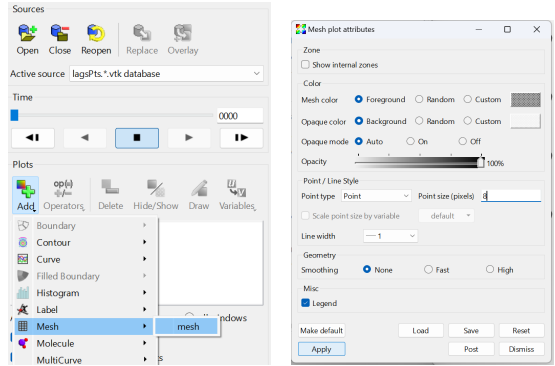
- **File** → **Open file** (or the folder icon), go to the run's `viz_IB2d` folder.
- With **File grouping** at **Smart**, the panel shows `lagsPts.*.vtk database (76 total files)` above the single files. Select the database line and **OK**.
- Nothing is drawn yet: the file is open, no plot exists. The **Active source** box in the main window now reads `lagsPts.*.vtk database`.
- The same works for `lagPtsConnect.*.vtk` (the points joined by the springs: slide 16) and for the force files in `hier_IB2d_data`.



The File open window with the grouped database selected (3.4.1, Windows).

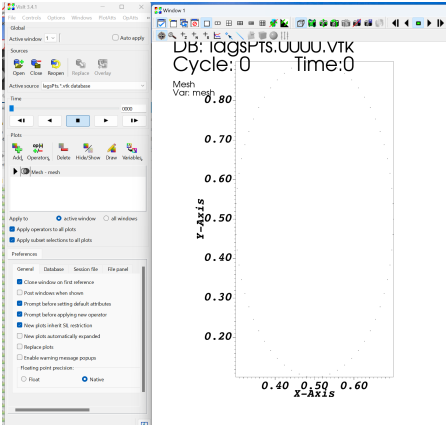
STEP 2: ADD THE MESH PLOT, DRAW IT, MAKE THE POINTS VISIBLE

- **Add** → **Mesh** → **mesh**, then **Draw**. The points appear, tiny.
- Double-click the **Mesh** entry in the plot list (or **PlotAtts** → **Mesh**): set **Point size (pixels)** to 8 and **Apply** (the practical's value; chapter 26 uses 4–8 for a figure). **Point type Point** draws squares, **Sphere** discs.
- The mesh colour can be set under **Mesh color** → **Custom**; black prints best.
- Headless, the same is `AddPlot("Mesh", "mesh")` with `MeshAttributes().pointSizePixels = 8` (`visit_ch4b.sh`).

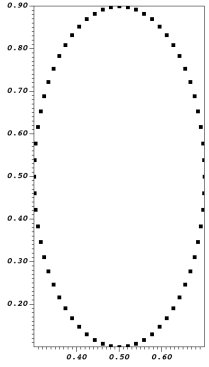


Left: **Add** → **Mesh**. Right: the Mesh attributes window with the point size.

THE POINTS, DRAWN



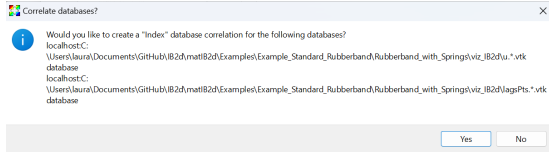
The 2024 window: the band's 64 points at $t = 0$ as first drawn, before the point size is set ("the vertices are small, so . . . "), with the default annotations (database name, cycle and time top left, user and date bottom right).



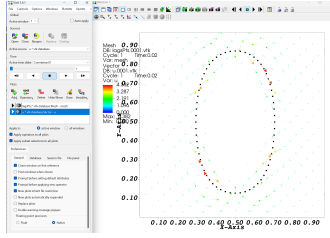
The same state from the Python run in VisIt 3.5.0 headless, annotations off except the axes. The time of a Python-written file is its index; the MATLAB window above says `Time: 0`.

STEP 3: THE VELOCITY VECTORS, AND THE CORRELATION QUESTION

- **Open** again and select `u.*.vtk database`. **Add** → **Vector** → **u**.
- VisIt asks whether to create an "Index" database correlation between `u.*.vtk database` and `lagsPts.*.vtk database`. **Yes**: from now on one time slider moves both, state k with state k. (**No** leaves two sliders, and the points would stay at $t = 0$ while the vectors advance.)
- **Draw**. At state 0 the velocity is zero, so nothing shows: advance with the ▷ button next to the time slider, or type a state number into it.
- The reset-view button (the camera with the green cross in the toolbar) shows the whole box again after a zoom or after the view has followed a plot.



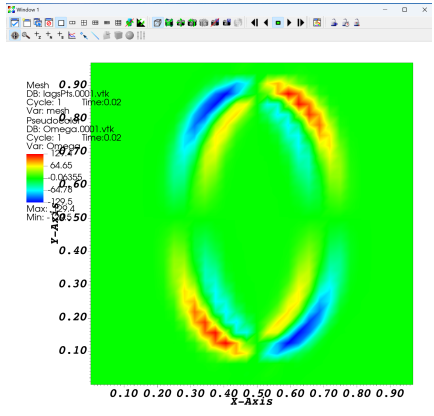
The prompt (3.4.1). Headless: `CreateDatabaseCorrelation("rb", (LAG, OM), 0)` and `SetActiveTimeSlider("rb")`; 0 is the index method.



The time controls and the reset-view button.

STEP 4: THE VORTICITY, AND PLOTS ON TOP OF PLOTS

- Select the `u` plot and **Hide**/**Show** it (the vectors are hard to read under a colour map at first). **Open** `Omega.*.vtk database`; **Add** → **Pseudocolor** → **Omega**; **Draw**; say **Yes** to the correlation again.
- The points have disappeared: VisIt draws plots in the order they were added, and the opaque colour map covers the earlier Mesh plot. Set the **Active source** back to `lagsPts.*.vtk database`, **Add** → **Mesh** → **mesh**, **Draw**: the points come back on top. (Or give the Pseudocolor an opacity below 1: next slide.)
- The default colour table (`hot`) and automatic limits change from state to state; the 2024 window at dump 1 reads ± 129.

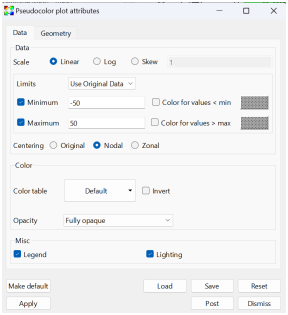


The 2024 window after **Draw**: the Pseudocolor of `Omega` at dump 1 (`Time: 0.02`, the MATLAB stamp), legend with automatic limits, the points hidden underneath.

STEP 5: MAKE THE VORTICITY READABLE

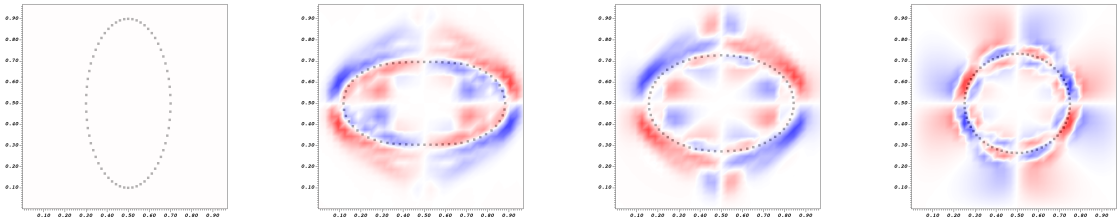
Double-click the Pseudocolor entry (or **PlotAtts** → **Pseudocolor**):

- **Limits**: **Minimum** -50, **Maximum** 50. The practical: “these are just approximate values”. They clip the peaks (129 at dump 1; above 50 in 57 of the 76 dumps) and show the shear layers at one scale, which is what makes frames comparable (chapter 26); the extremes at states 15 and 75 are ± 52 and ± 58 (the **MinMax** query, slide 17).
- **Centering**: **Nodal**. Harmless here: IB2d writes its fields as point data, which VisIt already treats as nodal (a headless render with and without the setting is pixel-identical); the step matters for IBAMR’s zone-centred vorticity, where it came from (chapter 26).
- **Opacity**: **Constant**, then 70 % on the slider (the 3.4.1 window shows the drop-down at **Fully opaque**), so that the vectors and points show through.



The Pseudocolor attributes window (3.4.1): limits, centering, opacity.

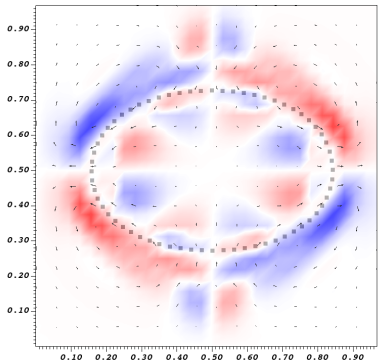
THE RESULT: FOUR STATES OF THE PYTHON RUN IN VISIT 3.5



States 0, 5, 15 and 75 ($t = 0$, 0.1, 0.3, 1.5 s): Pseudocolor of `Omega`, limits ± 50, nodal, 70 % opacity, `difference` colour table; the Mesh plot of the points added before it, which is why the points look grey through the colour map. Compare chapter 4a's matplotlib frames of the same files.

- At state 0 the field is zero everywhere (a uniform white): the fluid starts at rest and the band has not yet pulled.
- The four corner vortices and the thin shear layers along the band are the same at every resolution; their strength is not (chapter 4d).

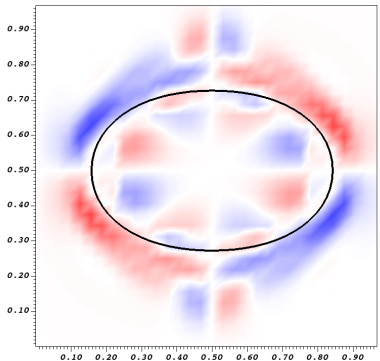
VECTORS AND COLOUR TOGETHER



State 15 with the Vector plot of `u` (600 vectors, uniform in space, scaled by magnitude) on top of the vorticity.

- The Vector plot's attributes (**Vectors**: uniformly located, a number such as 600 at 32^2 and 8000 at 512^2; **Scale** and **Scale by magnitude**; **Color**: constant) are chapter 26's; the practical leaves the defaults.
- VisIt 3.5 prints **The Vector plot of variable "u" yielded no data** when the plot is drawn at state 0 (the field is zero) and draws it anyway at the other states; chapter 26 notes the same warning. Not an error.
- The peak speed at state 15 is 1.59 (the **MinMax** query on the Vector plot); 4.5 over the whole run (chapter 4a).

THE CONNECTED POINTS



- `lagPtsConnect.*.vtk` holds the same points joined into line segments along the springs (and only the springs: without `springs = 1` no connection file is written, as `Wobbly_Beam`'s output shows): a Mesh plot of it with **Line width** 3 draws the band as a curve (left, state 15 over the vorticity), the choice for figures.
- Open it like `lagsPts`: `lagPtsConnect.*.vtk` `database`, **Add** → **Mesh** → **mesh**, line width in the Mesh attributes, and the same "Index" correlation.
- For an open structure (a beam, the heart tube's two walls) the segments follow the spring list, so a wall drawn this way shows gaps exactly where springs are missing: a second geometry check.

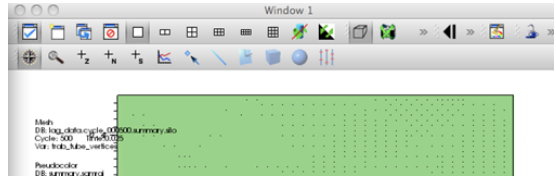
QUERIES: A NUMBER FROM THE WINDOW

- **Controls** → **Query** (chapter 26): **MinMax** on the active Pseudocolor gives the extremes with their node and position; on the Vector plot it gives the speed. At state 15: `Omega -- Min = -51.49 (node 411 at <0.84375, 0.375>)`, `Max = 52.21`; `u -- Max = 1.59 (node 516 at <0.125, 0.5>)`.
- **Pick** on a point (chapter 28) prints its coordinates and, for the force databases, the force; the points have no variables of their own in `lagsPts`.
- **Lineout** across the band gives a profile of `Omega` or `uMag` as a curve, with the time slider still working (chapter 26's Lineout; chapter 29 for the analysis).
- For a number against time (a tip's position, the peak force, a flow rate), chapter 33's readers do it in a few lines from the files; chapter 4a's half-width plot and the forces plot of slide 23 are that.

> The query must be made on the plot that holds the variable (select it in the plot list first); on the Mesh plot VisIt answers "Cannot perform a MinMax query on variable mesh."

ZOOM, ANNOTATIONS, SESSIONS

- Zoom: the magnifying glass in the toolbar, then drag a box in the window; the reset-view button restores the whole box. (The practical's sentence "the region of interest, namely that around the channel and cavities" is left over from the IBAMR deck it was adapted from.)
- Annotations (chapter 26, slide on annotations): **Controls** → **Annotation**: untick **Database** and **User information** for a clean figure; the **2D** tab for axis titles and fonts; the **Objects** tab to drop the legend's title and min/max and to move it.



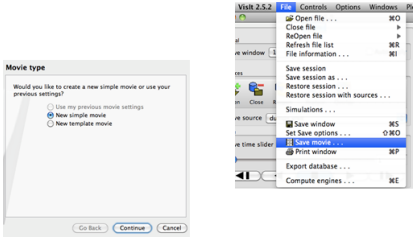
The zoom and reset-view buttons (a VisIt 2.5.2 window reused by the 2024 practical).

SESSIONS AND SAVING A WINDOW

- Sessions: **File** → **Save session** keeps every plot and setting; **Restore session** reopens them; **Restore session with sources** points the same session at another run's files (Lecture 10). Save one once the window looks right.
- Save window: **File** → **Save window** writes a PNG of the current state at the size set under **File** → **Set save options**.
- A session file is text (XML) and travels with the run: keep it next to `viz_IB2d` so that the figure can be remade after the next run.

A MOVIE

- The practical's route: **File** → **Save movie**, **New simple movie**, format **MPEG movie**, the → button, then the frame range, frame rate, stride, folder and name. It works; Lecture 10 calls it "not the best option".
- The better route (chapter 30): the same wizard with format **PNG** writes one frame per state; join them with ffmpeg (next slide).



The Save movie wizard's first two screens (2024 practical).

FRAMES AND FFMPEG

```
ffmpeg -framerate 10 -i rubberband%04d.png -pix_fmt yuv420p -vf "scale=trunc(iw/2)*2:trunc(ih/2)*2"
     rubberband.mp4
```

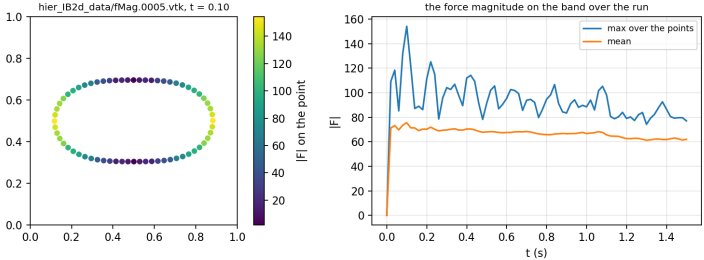
- Lecture 10 named “VirtualDub (free for PC) or Quicktime Pro” for the joining; ffmpeg is free on all three systems (`apt install ffmpeg`, Homebrew, or the Windows build) and scriptable.
- Headless, `SaveWindow()` in a loop over `SetTimeSliderState(k)` writes the frames without the GUI (chapter 32; `visit_ch4b.sh` saves four of them that way).
- The `-vf scale` part keeps both dimensions even, which the H.264 encoder requires; `-framerate 10` shows 76 frames in 7.6 s. Chapter 30 has the options (frame rate, size, a title frame, GIF).

THE SAME STEPS AS A SCRIPT (WHAT CHAPTER 32 TEACHES)

```
LAG = RUN + "/viz_IB2d/lagsPts.*.vtk database"
OM  = RUN + "/viz_IB2d/Omega.*.vtk database"
OpenDatabase(LAG)
AddPlot("Mesh", "mesh")
m = MeshAttributes(); m.pointSizePixels = 8; m.legendFlag = 0
m.meshColor = (0, 0, 0, 255); m.meshColorSource = m.MeshCustom;
      SetPlotOptions(m)
OpenDatabase(OM); ActivateDatabase(OM)
AddPlot("Pseudocolor", "Omega")
pc = PseudocolorAttributes(); pc.minFlag = pc.maxFlag = 1; pc.min, pc.max
       = -50, 50
pc.centering = pc.Nodal; pc.opacityType = pc.Constant; pc.opacity = 0.7
pc.colorTableName = "difference"; SetPlotOptions(pc)
CreateDatabaseCorrelation("rb", (LAG, OM), 0)   # the "Index" correlation
SetActiveTimeSlider("rb"); DrawPlots()
for k in (0, 5, 15, 75):
    SetTimeSliderState(k); DrawPlots()
    SetActivePlots(1); print(k, Query("MinMax"))
    s = SaveWindowAttributes(); s.fileName = "ch4b-visit-omega-%04d" % k
    s.family = 0; s.format = s.PNG; SetSaveWindowAttributes(s); SaveWindow
      ()
```

- Run with `visit -nowin -cli -s script.py` (no window) or `visit -cli -s script.py` (with one). This is how the pictures of slides 10–16 were made.
- Every button of the walk-through has a line here; **Controls** → **Command** records the lines as you click (chapter 32).
- The Mesh plot is added first and is plot 0; the Pseudocolor is plot 1, the one the query needs (`SetActivePlots(1)`).
- The full script, with the vectors and the connected points, is `wsl/final/visit_ch4b.sh`.

THE FORCES ON THE STRUCTURE: HIER_IB2D_DATA



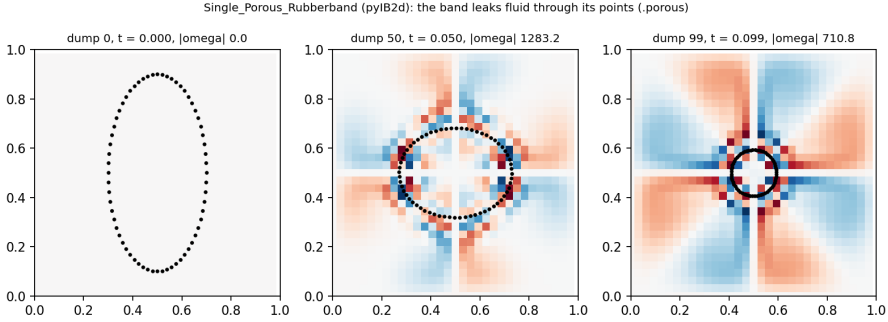
- `fMag.NNNN.vtk` in `hier_IB2d_data` is the magnitude of the force each point exerts on the fluid, as a scalar on the points; `fX_Lag`, `fY_Lag` are its components. Left: dump 5 ($t = 0.1$), when the force peaks (154 at the sides of the band, where the springs are most stretched); right: its maximum and mean over the points against time (the mean is 71 at the first dump, 76 at its peak and 62 at the end, as the band relaxes).
- In VisIt: open `fMag.*.vtk database`, **Add** → **Pseudocolor** → **fMag**, and the points colour by force (set the point size as before). From the files: chapter 33's `read_vtk_legacy` gives the points and the `fMag` array; the plot above is that, in Python.
- The sum of `fX_Lag` over the points is the net force on the fluid, the quantity chapter 11 computes for drag and lift in IBAMR; the same bookkeeping applies here.

FOUR MORE PYTHON EXAMPLES, RUN AS SHIPPED (OCTOBER 2026)

example (`pyIB2d/Examples`)	models (`input2d`)	grid, Δt, T	wall	output
`Wobbly_Beam`	beams + 2 target points (a curved beam held at both ends, straightening)	32^2, $5 \cdot 10^{-5}$, 0.05	2.7 s	100 dumps
`Single_Porous_Rubberband`	springs + porous points (`.porous`)	32^2, 10^{-4}, 0.1	3.2 s	100 dumps, 11 files each
`HeartTube_Muscle`	springs + beams + target points + muscles (`.muscle`, activation in `give_Muscle_Activation.py`)	128^2, $5 \cdot 10^{-4}$, 0.25	14 s	50 dumps
`Tracers_In_Impedance_Pump`	springs updated in time (`update_Springs.py`) + beams + target points + 110 tracers	64^2, 10^{-3}, 2	53 s (after a fix)	401 dumps, with `tracer` files

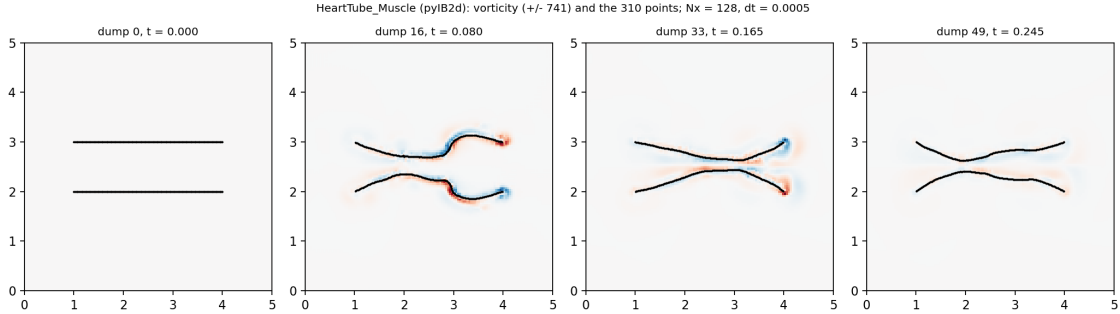
- All four print their models under “FIBER MODEL INCLUDES” (the heart tube: Springs, Beams, Target Pts, “MUSCLE MODEL (Force-Velocity / Length-Tension Model)”); none needs anything beyond chapter 4a’s set-up.
- `Rubberband_with_Beams` needs the `input2d` fix of chapter 4a before it does anything; the pump needs a one-line fix in the driver (slide 28).
- The VisIt steps are the same for every one of them; the pictures that follow are matplotlib on the files, as in chapter 4a.

THE POROUS BAND



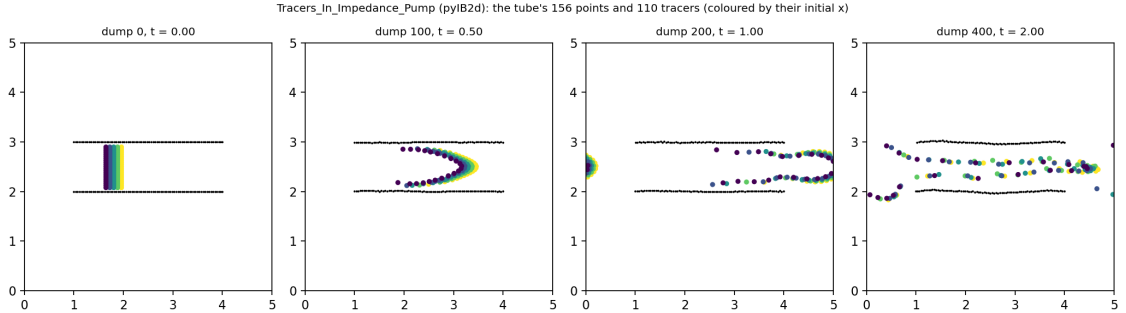
- `Single_Porous_Rubberband`: a 64-point band with a `.porous` file (a porosity coefficient per point) and springs 400 times stiffer than the springs example's ($k = 10^7$). Each point slips through the fluid in proportion to the force on it (`please_Compute_Porous_Slip_Velocity`), so the band leaks and shrinks fast (x half-width 0.09 by $t = 0.1$). Do not compare it with the springs run of chapter 4a (different stiffness); the MATLAB folder `Example_Porous_Rubberbands` has the side-by-side comparison with a sealed band at the same k.
- 1000 steps of 10^{-4} (the stiff springs set the time step; chapter 4d): 3.2 s, 100 dumps every 10 steps, 11 files per dump (`uX`, `uY`, `fX`, `fY`, `fMag` switched on in this `input2d`).

THE HEART TUBE WITH MUSCLES



- `HeartTube_Muscle`: two walls of a tube (310 points) held by target points at the ends, with springs and beams along them and muscle fibres across (`HeartTube.muscle`, the force-velocity/length-tension model of the 2015 paper); `give_Muscle_Activation.py` sends a wave of activation along the tube and the walls pinch in turn. `test_Activation_Wave.py` plots that wave without running anything.
- 128^2, 500 steps: 14 s, 50 dumps. The frames are scaled to ± 741, the largest value among the four shown; the run's peak is $3.5 \cdot 10^3$ in the first dump, a start-up transient on the walls. The muscle model, its parameters and what the tube pumps are chapter 4e's.

THE IMPEDANCE PUMP WITH TRACERS



- `Tracers_In_Impedance_Pump`: a tube of 156 points whose springs change their resting lengths in time (`update_Springs.py`, chapter 4c) to pinch one section periodically; 110 passive tracers (`HeartTube.tracer`) ride the flow and show the net transport: their mean x moves from 1.80 to 2.81 in 2 s, and that understates the transport, because tracers that reach the right edge re-enter at the left (the periodic box; visible in the last frame). The tube is 3 long in a box of 5. An impedance pump moves fluid without valves by pinching off-centre (chapter 4e; the IBAMR heart-tube pump is IB/11, chapter 9b; the two-point valveless pump is chapter 36).
- 64^2, 2000 steps: 53 s, 401 dumps every 5 steps, 12 files each (the `tracer.NNNN.vtk` files are the tracers' positions; VisIt opens them like the points).

THE PUMP EXAMPLE AS SHIPPED STOPS AT THE FIRST STEP

In the 2024 clone, `python3 main2d.py` in `Tracers_In_Impedance_Pump` writes dump 0000 and dies:

```
  File ".../IBM_Blackbox/IBM_Driver.py", line 821, in main
    xT, yT = please_Move_Lagrangian_Point_Positions(Uh, Vh, xT, yT
      , xT, yT,
TypeError: please_Move_Lagrangian_Point_Positions() missing 4
      required positional arguments: 'porous_Yes', '
      poroelastic_Yes', 'poroelastic_info', and 'F_Poro'
```

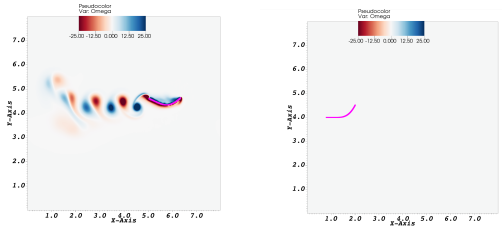
The tracer branch of the driver calls the point mover with the signature it had before the porous and poroelastic arguments were added (and without `mu`); the two calls for the structure itself were updated, this one was not. Any Python example with `tracers = 1` fails the same way.

The fix, in `pyIB2d/IBM_Blackbox/IBM_Driver.py` at that line (the tracers are never porous):

```
xT, yT = please_Move_Lagrangian_Point_Positions(mu, Uh, Vh, xT,
       yT, xT, yT,
    x, y, dt, grid_Info, 0, 0, 0, 0)
```

- With it the example runs to the end (53 s) and the tracers move as on the previous slide.
- Like the `nonInv_beams` key of chapter 4a and the `fNorm`/`fTan` test, this is worth a note to the IB2d repository; the MATLAB version has no such problem (its driver passes the full argument list).
- The lesson is chapter 4a's: a run that starts is not a run that finished. Count the dumps.

THE SWIMMER (MATLAB): A BEAM THAT CHANGES ITS CURVATURE



Start and end of the run (2024 practical).

- `matIB2d/Examples/Examples_First_Year_Seminar/Swimmer`: 512^2 on an 8×8 box, $\Delta t = 2.5 \cdot 10^{-5}$, $T = 3$. "about 15-20 mins to get to time 1" (the practical); run it, watch, and stop it with Ctrl-C when you have seen enough.
- A line of points with springs and non-invariant beams whose preferred curvature changes in time: the swimming comes from the fluid. Chapter 4c builds a smaller version of exactly this mechanism.

THE SWIMMER'S FILES

The files, and what each does (the practical's slides 31–33):

- `Swimmer_Geometry.m` writes `swimmer.vertex`, `.spring`, `.nonInv_beam`: a line of points with springs and non-invariant beams (chapter 4a).
- `swimmer.phases` stores the x and the two y profiles of the swimmer's two phases; `test_Script.m` reads it and displays the motion without running the fluid.
- `update_nonInv_Beams.m` is called every step (`update_nonInv_beams = 1`): it takes the time within the period ($t =$ `rem(current_time, period)`, period 0.5 s as a 0.25 s down-stroke and a 0.25 s up-stroke), interpolates between the two phases with a piecewise quadratic–cubic–quadratic blend (coefficients `a`, `b`, `c`, `d`, `g`, `h` in the file), computes the curvatures as second differences, and writes the new C_x, C_y into columns 5 and 6 of the beam table.
- The curvature is prescribed, the position is not: the beam's forces and the fluid decide where the swimmer goes.

THE LEAF (MATLAB): A STRUCTURE THAT RELEASES A SCALAR INTO A FLOW



Start and end (2024 practical): the leaf on its stem in the channel, the concentration it sheds.

- `matIB2d/Examples/Example_Concentration_Dynamics/Leaf_64`: a 384×64 grid on a 30×5 box, $\Delta t = 5 \cdot 10^{-5}$, $T = 3.25$; a cylinder (the stem) held by target points with a flexible line (the leaf) attached, built from `leaf.vertex`, `.spring`, `.beam`, `.mass`, `.target`, plus `leaf.concentration`, the initial scalar field.

THE LEAF'S FILES

- `Leaf_Leaf.m` writes the files; `give_me_source_model.m` is the source of the scalar on the leaf; `please_Compute_External_Forcing.m` sets the background flow entering on the left (`arb_ext_force = 1`: the "pseudo-inflow" of chapter 4a).
- The practical calls the scalar heat on one slide and oxygen on the next; in the code it is a passive concentration (the 2022 paper), and either reading is fine.
- A 384×64 run of 65 000 steps is hours in MATLAB; it is here to be read and to show what the concentration machinery does. The IBAMR counterpart with advection–diffusion is chapter 24.

WHERE TO GO FROM THESE EXAMPLES

- Chapter 4c (moving boundaries): the update functions these examples use (`update_Springs`, `update_Target_Point_Positions`, `update_nonInv_Beams`) written from scratch on the sawtooth channel, and your own geometry.
- Chapter 4d (stability and convergence): what happens to the rubber band when Δt doubles or the stiffness is multiplied by ten (the "BLOW UP" message), and the same run at 32^2, 64^2 and 128^2.
- Chapter 4e (the gallery, the muscle models): the heart tube's muscle law, the pumps, the mass-and-gravity races, the porous bands, the Boussinesq and concentration examples, with the pictures and movies of the 2016 talk.
- Chapter 4f (MeshmerizeMe): a drawing into `.vertex` files.
- Part V for VisIt beyond this walk-through: chapter 26 (making it look better), 28 (the points and their variables), 29 (data analysis), 30 (movies), 31 (VisIt on the cluster), 32 (the command line and Python), 33 (the files in Python and MATLAB).

EXERCISES

1. The practical's two: change the band so that it is stretched in y instead of x ("enough that you can see it"), and change the viscosity in `input2d` to put the band in a very viscous fluid. Plot the initial state and the first dump after it with the points, the vectors and the vorticity (homework 4 of Lecture 11), and say in two sentences how the viscous run differs.
2. Save a VisIt session of the finished window; restore it with sources on the viscous run's folder.
3. Make a 76-frame movie of the vorticity with fixed limits, as PNG frames joined by ffmpeg; note the file size against the wizard's MPEG.
4. From `hier_IB2d_data`, plot the maximum of `fMag` against time for the springs band and for the damped-springs band of chapter 4a on the same axes.
5. Run `HeartTube_Muscle` and open it in VisIt: a Pseudocolor of `uMag` with the connected points; find the time of the strongest pinch from the **MinMax** query over the states.
6. Apply the one-line fix of slide 28 and run the pump; with chapter 33's reader, plot the mean x of the tracers against time, unwrapping the jumps of tracers that cross the periodic edge, and read off the pumping speed.

SUMMARY

- A run is `main2d` in an example folder; MATLAB shows a live figure, Python can; both leave `viz_IB2d` (fields, points) and `hier_IB2d_data` (forces), one file per dump and variable.
- In VisIt, open the grouped database (`lagsPts.*.vtk database`), add a Mesh plot, draw, set the point size; open `u.*.vtk` and `Omega.*.vtk` the same way, accept the "Index" correlation so that the sliders move together, add the Vector and Pseudocolor plots; re-add the points on top; fixed limits, nodal, 70 % opacity; play, zoom, save a window or frames.
- MATLAB's files carry a time stamp, Python's an index; everything else about the files is the same in the two versions.
- The forces on the points are scalars on the same point files; one number per dump, plotted against time, is the usual result.
- Four Python examples run in seconds to a minute (one after a one-line fix); the swimmer and the leaf are MATLAB examples to read for the update functions and the concentration machinery.
- Next: chapter 4c moves the boundaries.

REFERENCES AND CREDITS

- The IB2d repository, https://github.com/nickabattista/IB2d: `pyIB2d/Examples/{Rubberband_with_Springs, Wobbly_Beam, Single_Porous_Rubberband, HeartTube_Muscle, Tracers_In_Impedance_Pump}`, `matIB2d/Examples/Examples_First_Year_Seminar/Swimmer`, `matIB2d/Examples/Example_Concentration_Dynamics/Leaf_64`, the writers in `IBM_Driver.m/.py` (`print_vtk_files`, `savevtk_*`); the README's VisIt note for macOS. Code as of the 2024-03-06 clone.
- VisIt 3.5.0 (https://visit-dav.github.io/visit-website/): the headless script `wsl/final/visit_ch4b.sh` and the metadata check `ch4b_facts.sh`; the runs `wsl/final/ib2d_pyrun3.sh`, `ib2d_pump2.sh`; the figures `tools/ch4b_figures.py`, `ch4b_figures2.py`, with `examples/analysis/read_vtk_legacy.py`.
- Sources merged (slide 3): "IB2d Practical" (2024; its 25 screenshots are reused here as `ch4b-ibp-*.png`: VisIt 3.4.1 on Windows, with a few older VisIt 2.x windows among them); "ib2d part 2" (Lecture 10, 2021, edited 2023); "A quick intro to ib2d" (Lecture 9); N. Battista's video tutorial 4. The 2015, 2017, 2018 and 2022 IB2d papers (chapter 4a's list) for the muscle, porous, tracer and concentration machinery.
- FFmpeg (https://ffmpeg.org), version 6.1 in the WSL used here.